

Полный гайд по Turbo Intruder

Turbo Intruder — это расширение для Burp Suite, предназначенное для отправки большого количества HTTP-запросов и анализа ответов. Его основное назначение — запуск длительных, сложных или сверхбыстрых атак, которые стандартный Intruder (особенно в бесплатной версии Burp) не сможет выполнить.



В чём его сильные стороны

Быстрый

Turbo Intruder использует самописный HTTP-стек, оптимизированный под скорость. Благодаря этому он часто работает быстрее, чем популярные асинхронные скрипты на Go.

Масштабируемый

Инструмент может использовать почти постоянное количество памяти, что делает его устойчивым при атаках, рассчитанных на миллионы запросов. Он также может работать в headless-режиме (без интерфейса) через командную строку.

Гибкий

Атаки описываются с помощью Python-скриптов, что даёт полный контроль: можно работать с подписанными запросами, делать многошаговые атаки, обрабатывать нестандартные ответы. Собственный HTTP-движок также позволяет отправлять "ломанные" или нестандартные запросы.

Удобный

Умеет автоматически фильтровать «скучные» ответы с помощью алгоритма сравнения, заимствованного из Backslash Powered Scanner. Это помогает быстро выделить интересные результаты даже при больших атаках.



Ограничения

- Освоение требует времени — интерфейс и логика работают через Python.
- Его сетевой стек не такой стабильный и проверенный, как у Burp.
- Заточен под работу с **одним хостом**. Если нужно тестировать множество сайтов — лучше использовать [ZGrab](#).



Как начать

1. Установка

В Burp Suite:

Extender → BApp Store → Turbo Intruder → Install

2. Альтернативный способ

Сборка из исходников:

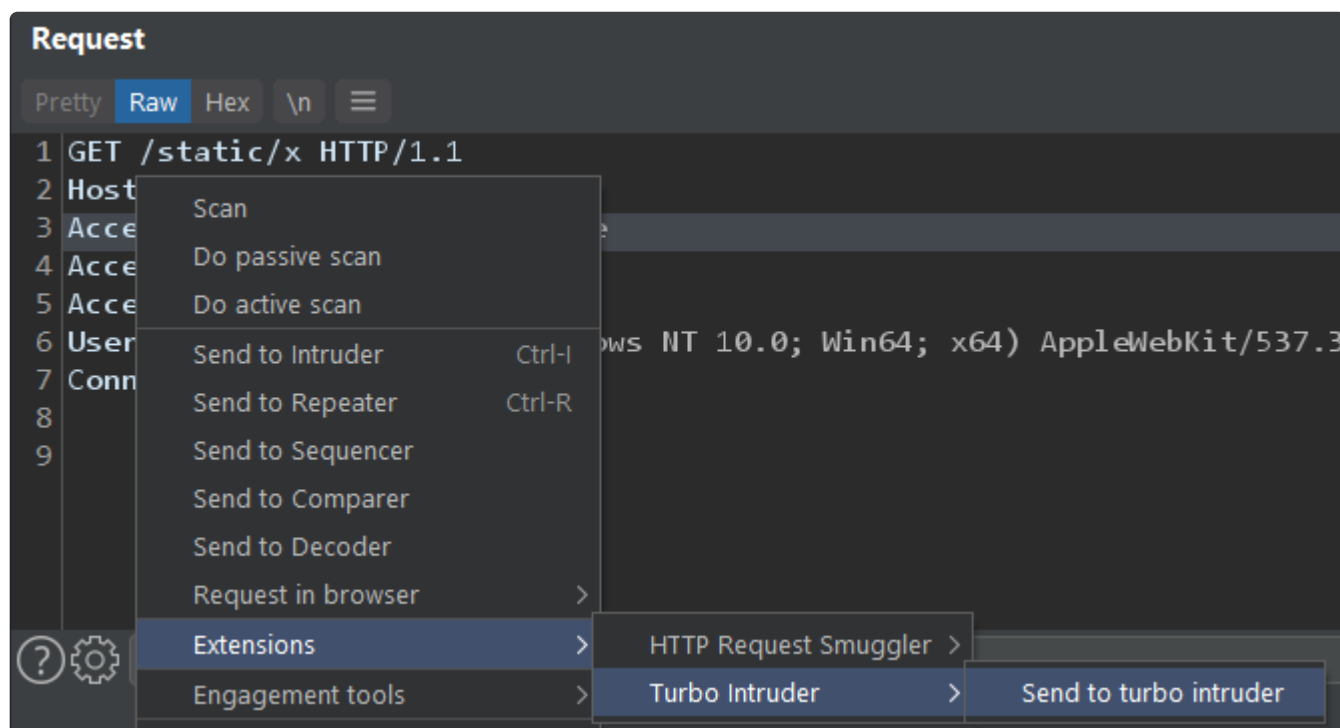
```
gradle build fatJar
```

Загрузка JAR-файла:

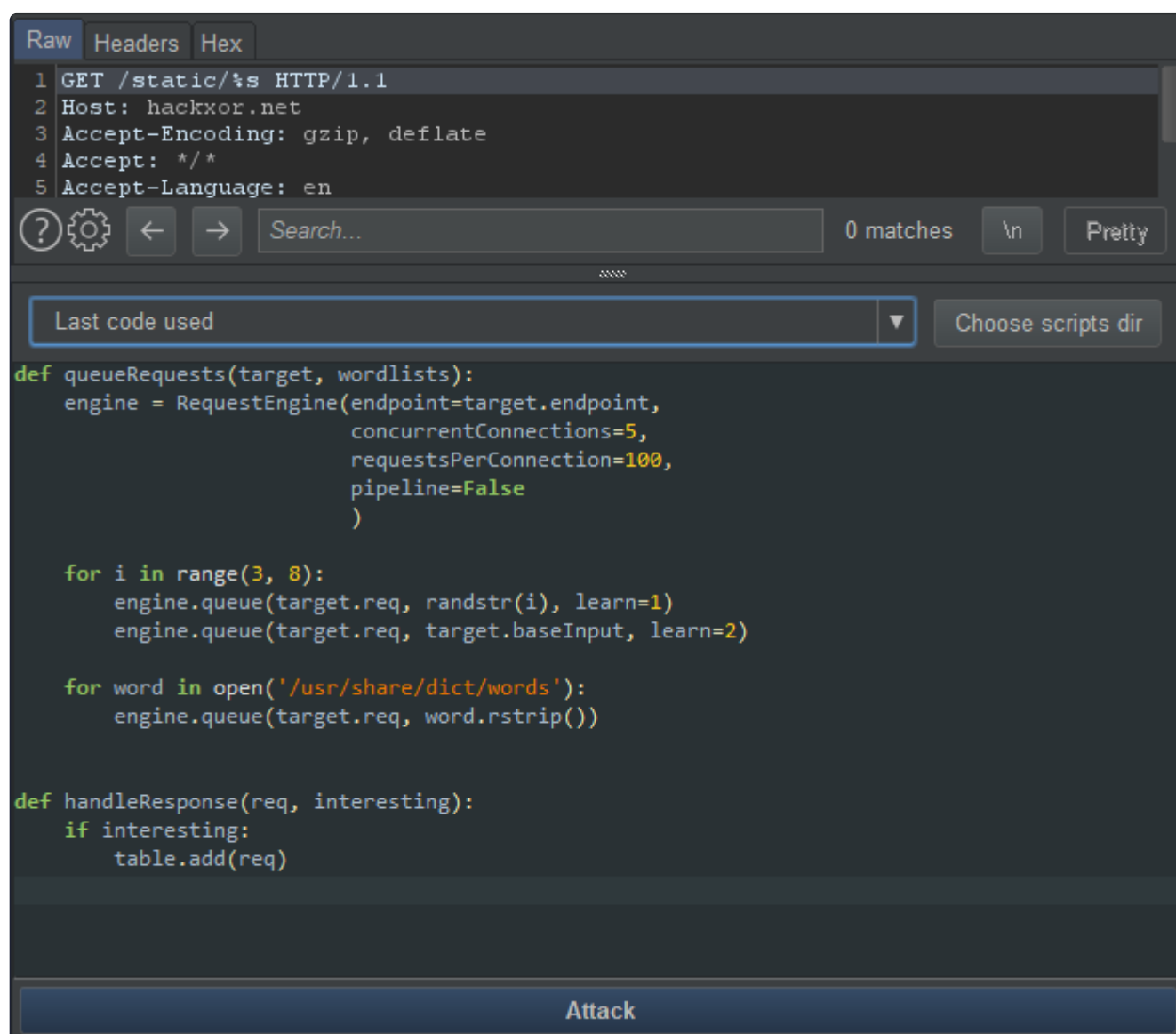
Extender → Extensions → Add → выбираешь turbo-intruder-all.jar

3. Использование

- Выделяешь в Burp интересующий параметр или часть запроса
- Правый клик → **Send to Turbo Intruder**



Откроется окно с шаблонным скриптом Python и запросом:



В скрипте будет `%s` — туда подставляются payload'ы. Запускаешь атаку — `Ctrl+Enter`.



Основы написания скриптов

Пример базового шаблона:

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=10,
                           requestsPerConnection=100,
                           pipeline=False)
```

```
for word in wordlists.words:
    engine.queue(target.req, word)

def handleResponse(req, interesting):
    if '200 OK' in req.response:
        table.add(req)
```

- `%s` в теле запроса заменяется на `word` из `wordlist`
- В `handleResponse()` ты решаешь, что считать «интересным» ответом и добавлять в таблицу
- Можно фильтровать по статусу, размеру, ключевым словам, времени отклика и т. д.



Подключение своих словарей и шаблонов

- Вместо `wordlists.words` можно использовать любой файл:

```
for word in open('my_wordlist.txt'):
    engine.queue(target.req, word.strip())
```

- Шаблоны скриптов можно сохранять отдельно и загружать вручную через редактор Turbo Intruder.



Фильтрация результатов

По умолчанию **ничего не отображается**, пока ты явно не добавишь ответ в таблицу:

```
def handleResponse(req, interesting):
    if 'admin' in req.response:
        table.add(req)
```

Можно использовать **декораторы**:

```
@MatchStatus(200,204)
@MatchSizeRange(100,1000)
def handleResponse(req, interesting):
    table.add(req)
```

Также можно использовать `interesting`, если включена "обучающая" атака с `learn=True` — в этом случае Turbo Intruder сам фильтрует скучные ответы.



Ускорение атаки

1. **Удаляй лишние заголовки и куки** — каждый байт имеет значение при десятках тысяч запросов
2. **Используй HEAD-запросы или заголовок Range**, если тебе не нужно тело ответа
3. **Выбери лучший движок**:
 - `Engine.HTTP2` — самый быстрый (если сервер поддерживает)
 - `Engine.THREADED` — по умолчанию, хорошая скорость
 - `Engine.BURP2` или `Engine.BURP` — для стабильности и прокси
4. **Настрой параметры движка**:

```
engine = RequestEngine(  
    endpoint=target.endpoint,  
    engine=Engine.THREADED,  
    concurrentConnections=20,  
    requestsPerConnection=100,  
    pipeline=True  
)
```

5. Минимизируй нагрузку на UI:

- `table.add()` — затратен для интерфейса. Добавляй в таблицу только нужные ответы
- Избегай `regex`, строковых циклов и длинных логик в `handleResponse`



Долгие атаки и производительность

Если запускаешь атаку на миллионы запросов:

- Не добавляй в таблицу все ответы
- Используй `for line in open('...')`, а не `.readlines()`, чтобы не держать весь словарь в памяти



Поиск race condition

По умолчанию Turbo Intruder делает **стриминговую атаку** — запросы летят по мере обработки. Для race condition нужно **синхронно отправить пачку** — используешь систему `gate`, как в `race.py`.



Встроенные словари

Есть два встроенных:

- Большой словарь для брутфорса
- Словарь, составленный из слов, найденных в прокси-трафике

Использование показано в `specialWordlists.py`.



Использование в командной строке

Turbo Intruder можно запустить как `jar` без Burp Suite:

```
java -jar turbo.jar <script.py> <request.txt> <endpoint> <baseInput>
```

Пример:

```
java -jar turbo.jar examples/basic.py examples/request.txt https://example.net:443 foo
```

Важно: фильтрация интересных ответов не работает без Burp, потому что она использует его API.



Поддержка нескольких параметров

Можно передавать несколько значений, передавая список:

```
engine.queue(target.req, [value1, value2])
```

Работа с несколькими хостами

Один движок = один хост. Если нужно атаковать разные сайты — создавай разные движки.

Хранение состояния

Для сложных атак можно сохранять состояние между запросами:

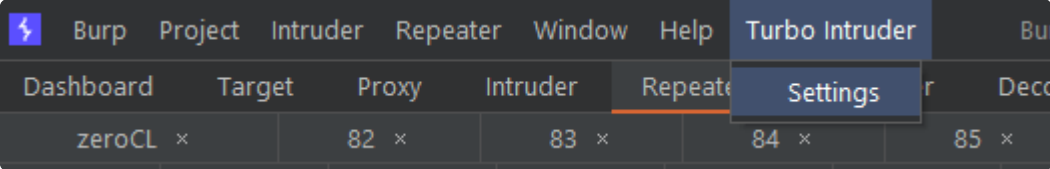
```
engine.userState['username'] = 'admin'
```

Примеры

Большое количество примеров (race condition, фильтрация, подстановки, тайминги и т.д.) смотри в папке `resources/examples` или на [GitHub Turbo Intruder](#).

Настройки редактора

Можно включить номера строк, изменить размер шрифта и др. через настройки редактора:



Заключение

Turbo Intruder — это инструмент для тех, кто хочет контролировать процесс атаки на 100%.

Он даёт возможности, которые недоступны в Intruder Community и даже частично в Pro. Особенно он полезен в ситуациях, где требуется:

- высокая скорость (до десятков тысяч RPS)
- сложная логика подстановки
- фильтрация по ключевым признакам
- кастомные сценарии, такие как race condition

Инструмент требует понимания Python и сетевых взаимодействий, но с его помощью можно реализовать почти любую идею в рамках HTTP-атаки.